

Audit technique approfondi - IPELAN Mobile

React Native (Expo SDK 54) ↔ Moodle 4.4.3

Analyse automatisée

24 avril 2026

Table des matières

Audit technique approfondi - IPELAN Mobile	1
1. Verdict en une page	1
2. Méthodologie	2
3. Bugs critiques (priorité P0 - bloquants)	2
4. Bugs importants (priorité P1)	4
5. Couverture API Moodle 4.4.3 - gaps fonctionnels	5
6. Fonctionnalités manquantes	7
7. Architecture & dette technique	9
8. Sécurité	9
9. Roadmap priorisée	10
10. Annexes	11
11. Conformité au cahier des charges	12

Audit technique approfondi - IPELAN Mobile

Application mobile éducative IPELAN - apprentissage des langues nationales mauritaniennes (Pulaar, Soninké, Wolof) - React Native / Expo SDK 54 ↔ backend Moodle 4.4.3.

Cet audit est le résultat d'une analyse statique du dépôt à la date du 24 avril 2026 (commit en cours, branche master). Il couvre l'ensemble du périmètre demandé : bugs concrets et vérifiables côté React Native, défauts d'intégration avec l'API Moodle 4.4.3, fonctionnalités incomplètes ou totalement absentes, et roadmap priorisée pour rendre l'application *production-ready*.

L'auteur de la documentation interne (RAPPORT.md, SYNTHESE_EXECUTIVE.md) avait déjà tiré la sonnette d'alarme : « 70 % des activités sont statiques » et « 30 % d'intégration Moodle ». Le présent audit confirme et précise - fichier par fichier, ligne par ligne - la nature exacte de cette dette.

1. Verdict en une page

L'application IPELAN se présente comme une plateforme hors ligne synchronisée avec Moodle. En réalité elle se comporte comme une coquille mobile **partiellement connectée** : l'authentification, le listing de cours et la lecture des contenus EPUB fonctionnent, mais la chaîne pédagogique critique (récupération de questions, soumission de résultats, persistance de la progression dans Moodle, classement) souffre soit d'erreurs d'implémentation, soit de fonctions absentes, soit de stubs masqués.

Trois constats majeurs se dégagent :

1. **Le projet ne compile pas en l'état.** Le compilateur TypeScript signale trois erreurs de syntaxe bloquantes dans `services/api/wordOrderService.ts` (un bloc de code dupliqué laissé

hors scope). Tant que ces erreurs ne sont pas corrigées, aucun build de production n'est techniquement réalisable.

2. **La sécurité côté serveur est compromise par un token administrateur Moodle exposé** dans la variable `EXP0_PUBLIC_MOODLE_TOKEN`. Le préfixe `EXP0_PUBLIC_` injecte cette valeur dans le bundle client : tout APK distribué fournit, par décompilation triviale, un accès admin total à `https://moodle.richatt.com`.
3. **La synchronisation Moodle est silencieusement défaillante** sur deux points clés : la fonction `submitGradeToMoodle` est appelée avec un `wstoken` vide, et la lecture de token côté `progressSync.ts` cible une clé `SecureStore` (`auth_token`) qui n'est jamais écrite (la bonne clé est `moodle_token`). L'application affiche "Synchronisé" alors que rien n'est persisté côté Moodle.

À cela s'ajoutent une absence totale d'i18n (l'application enseigne trois langues mais son interface est figée en français), aucun support d'accessibilité (zéro `accessibilityLabel` dans 14 000 lignes), un mode hors ligne approximatif (détection de réseau via `fetch('google.com/favicon.ico')` plutôt que `NetInfo`), et un écran de réinitialisation du mot de passe qui simule sa réussite avec `setTimeout` sans jamais contacter Moodle.

Estimation : **4 à 6 semaines de travail à un développeur senior** pour amener le projet à un état véritablement *production-ready*, dont une semaine est strictement bloquante (compilation, sécurité, sync).

2. Méthodologie

L'audit couvre l'arbre suivant :

- 39 écrans (`app/**/*.tsx`)
- 5 composants partagés (`components/`)
- 30 services (`services/`, dont 14 sous `services/api/`)
- 17 hooks (`hooks/`)
- Configuration Expo (`app.json`, `babel.config.js`, `metro.config.js`, `.env`)
- 14 192 lignes de code TypeScript/TSX au total

Les techniques utilisées :

- Lecture exhaustive des fichiers de service Moodle et des hooks de chargement d'activité.
- Compilation TypeScript stricte (`tsc --noEmit`).
- Recherche par expressions régulières des marqueurs de dette technique (TODO, FIXME, mock, hardcoded, `console.error`, etc.).
- Recoupement des `wsfunction` invoquées avec la liste officielle des web services Moodle 4.4.3.
- Vérification des règles internes du projet (`AGENTS.md`).

Toutes les références dans ce rapport pointent vers des fichiers et lignes réels du dépôt. Les comportements décrits sont vérifiés à la source - aucun n'est extrapolé.

3. Bugs critiques (priorité P0 - bloquants)

3.1 Erreur de compilation TypeScript dans `wordOrderService.ts`

`services/api/wordOrderService.ts` contient un bloc de code dupliqué laissé en dehors de la boucle `for`. La variable `result y` est référencée hors scope.

```
services/api/wordOrderService.ts(84,5): error TS1472: 'catch' or 'finally' expected.
```

```
services/api/wordOrderService.ts(85,5): error TS1005: 'try' expected.
```

```
services/api/wordOrderService.ts(89,1): error TS1128: Declaration or statement expected.
```

Ces trois erreurs apparaissent à la première exécution de `npx tsc --noEmit`. Le projet n'est pas compilable en l'état. Les lignes 64 à 83 du fichier reproduisent un bloc de logique de parsing déjà présent lignes 31 à 64 et constituent du code mort syntaxiquement invalide.

Correction : supprimer purement et simplement les lignes 65 à 83 du fichier (le second `for (const page of pages)`), puis vérifier que `npx tsc --noEmit` retourne zéro erreur.

3.2 Token admin Moodle exposé dans le bundle client

Le fichier `.env` contient `EXP0_PUBLIC_MOODLE_TOKEN=3dd356c41b8534c0d7a404df0608f7ac` - un token admin réel. Le préfixe `EXP0_PUBLIC_` est documenté par Expo comme **injecté dans le bundle JavaScript** distribué au client. Toute personne qui télécharge l'APK peut le récupérer en moins de cinq minutes (`apktool`, `strings`, `grep "wstoken"`).

Treize fichiers de l'app utilisent ce token comme *fallback admin* pour contourner les permissions d'élève (`hooks/useCourses.ts:36`, `services/api/courseService.ts:4`, `services/api/badgeService.ts:3`, `services/api/moodleAuth.ts` lignes 14, 73, 94, 102, 125, etc.). Cela signifie que la plate-forme Moodle ne s'appuie en pratique pas sur l'authentification réelle de l'utilisateur pour servir les contenus pédagogiques.

Conséquences directes :

- Lecture/écriture intégrale du contenu Moodle par n'importe quel utilisateur de l'APK.
- Modification du profil de n'importe quel élève (champs custom `ipelan_xp`, `ipelan_streak`, `ipelan_coins`).
- Inscription/désinscription en masse (`enrol_manual_enrol_users`, `auth_email_signup_user`).
- Export de données personnelles (RGPD).

Le `.gitignore` couvre bien `.env` (ligne 54), donc le secret n'est pas dans Git, mais il sera dans **chaque APK livré**. Solution : déplacer toutes les opérations privilégiées vers un proxy serveur (Cloud Run, Firebase Functions, ou un endpoint PHP côté Moodle) qui valide le token utilisateur avant de relayer la requête. Le token admin ne doit **jamais** quitter le serveur.

3.3 submitGradeToMoodle envoyé avec un wstoken vide

Dans `services/api/userProgressService.ts`, la fonction `submitGradeToMoodle` (ligne 261) construit l'appel suivant :

```
await moodleFetch('/webservice/rest/server.php', {
  wstoken: '', // ← VIDE
  wsfunction: 'core_completion_update_activity_completion_status_manually',
  ...
});
```

Aucun token n'est passé en paramètre, et celui de l'utilisateur n'est pas récupéré non plus. Moodle renvoie systématiquement `invalidtoken`. Pire, le `try/catch` qui entoure l'appel intercepte l'erreur et **retourne true** (ligne 287, branchement `invalidparameter/Valeur incorrecte` qui couvre par accident les exceptions `invalidtoken`). L'utilisateur voit donc toujours « Synchronisé », alors que la complétion n'est jamais envoyée à Moodle.

Correction : ajouter `token: string` au paramètre, puis logger explicitement l'erreur sans la noyer.

3.4 Désynchronisation des clés SecureStore (auth_token vs moodle_token)

`services/storage/tokenStorage.ts` enregistre le token sous la clé `moodle_token` (constante `TOKEN_KEY` ligne 4). Mais `services/sync/progressSync.ts:37` lit avec `SecureStore.getItemAsync('auth_token')` - clé qui n'est jamais écrite ailleurs dans le projet.

Conséquence : `getStoredToken()` renvoie toujours `null`. Toute branche qui s'appuie dessus échoue silencieusement. Plusieurs branches du fichier `progressSync.ts` lisent des appels Moodle qui n'aboutiront jamais à un appel HTTP réel.

Correction : remplacer 'auth_token' par 'moodle_token' (ou importer la constante depuis tokenStorage.ts), et homogénéiser une seule source de vérité.

4. Bugs importants (priorité P1)

4.1 Reset password est un mock (app/(auth)/resetpassword.tsx:24)

```
// TODO: Connect to Moodle API for password reset
// await requestPasswordReset(email);
setTimeout(() => {
  setLoading(false);
  setStep(2);
}, 1000);
```

Le bouton « Envoyer le lien » fait juste avancer le wizard et fait croire à l'utilisateur qu'un email a été envoyé. Aucun appel `core_auth_request_password_reset` n'est fait. Le formulaire d'étape 2 demande un nouveau mot de passe et le « valide » avec un autre `setTimeout`. Le mot de passe n'est **jamais** réellement changé sur Moodle.

4.2 ListeningExercise détourne mod_choice

L'écran d'écoute s'appuie sur `mod_choice` (un module Moodle conçu pour les *sondages*, sans notion de réponse correcte). `services/api/listeningService.ts:60` sélectionne donc une option arbitraire comme « bonne réponse », et `hooks/useListening.ts:165-177` génère le `correctIndex` en mélangeant aléatoirement les distracteurs.

Le résultat est pédagogiquement faux : la « bonne » réponse n'a aucune réalité côté Moodle, l'enfant peut être noté faux pour une réponse en réalité valide. Pour une vraie activité d'écoute, il faut soit un module `mod_quiz` à question unique avec audio attaché, soit un type d'activité dédié (un plugin Moodle local).

4.3 processSingleAnswer envoie data comme objet et non comme paramètres aplatis

`services/api/quizService.ts:241-247` passe :

```
data: [
  { name: `q${slot}:_sequencecheck`, value: String(sequencecheck) },
  { name: `q${slot}:_answer`, value: answerValue },
],
```

Le code aplatisseur de `moodleClient.ts:21-32` aplatit les objets imbriqués mais traite les tableaux comme des objets indexés numériquement, ce qui fonctionne par chance - sauf que d'autres appels du même fichier (`saveQuizAnswers`, `finishQuizAttempt`) aplatissent eux-mêmes manuellement. Deux conventions cohabitent : à terme, l'une va casser. À uniformiser.

4.4 Boucle de splash bloquante (app/index.tsx:67)

```
const timer = setTimeout(() => { ... }, 5000);
```

L'écran d'accueil attend 5 secondes même si l'authentification a déjà été restaurée. C'est imposé pour laisser jouer les animations, mais c'est ressenti comme une latence par les utilisateurs. À transformer en *gate* qui navigue dès que `prepareApp` est terminé, avec un minimum d'1 seconde plutôt que 5.

4.5 Math.random() - 0.5 pour le shuffle (8 occurrences)

Le pattern `array.sort(() => Math.random() - 0.5)` apparaît dans `app/(stacks)/(cours)/game.tsx:53,93`, `hooks/useListening.ts:165,168`, `hooks/useActivityContent.ts:531`. Cet algorithme produit une

distribution **biaisée** (les éléments en début de tableau ont plus de chances de rester en début). Pour une appli pédagogique où les options de réponse doivent être perçues comme équitablement mélangées, c'est mesurable et perceptible par un enseignant. Remplacer par Fisher-Yates (déjà implémenté dans `services/api/wordOrderService.ts:11-17` et `app/(stacks)/(cours)/association.tsx:48`).

4.6 Audio simulé en l'absence de fichier (`listening.tsx:67-68`, `dictation.tsx`)

```
} else {  
  setIsPlaying(true);  
  setTimeout(() => setIsPlaying(false), 2000);  
}
```

Si Moodle ne renvoie pas d'audio, l'app fait simplement clignoter le bouton 2 s. L'enfant croit avoir entendu un son et donne sa réponse au hasard. Mieux vaut afficher un *empty state* explicite (« audio non disponible, signaler ») que tromper l'utilisateur.

4.7 Onboarding - faute de frappe figée

`app/(auth)/onboarding.tsx:35` : "Des étoiles, des badges et desXP pour chaque activité réussie !" (manque l'espace avant « XP »). Faute exposée à tous les premiers utilisateurs.

4.8 Violations des règles internes AGENTS.md

Le manifeste agent du projet impose **NativeWind/Tailwind exclusivement** - pas de `StyleSheet.create()`. Cinq fichiers violent cette règle :

- `app/(auth)/login.tsx`
- `app/(auth)/signup.tsx`
- `app/(auth)/resetpassword.tsx`
- `components/PolicyModal.tsx`
- `components/epub/EPUBLessonViewer.tsx`

Cette incohérence rend la maintenance du thème (couleurs primaires #002366, #4a90e2) plus difficile : un changement de palette demanderait de toucher à la fois `tailwind.config.js` et chaque `StyleSheet`.

4.9 useSignup casse l'inscription par email (`hooks/useSignup.ts:14`)

```
const { username, email, password, firstname, lastname, city } = formData;  
if (!username) throw new Error("Username is required");
```

Or `app/(auth)/signup.tsx` ne dispose d'**aucun champ username** dans son formulaire. Le `signup` lève donc systématiquement l'erreur - ou alors un `username` est dérivé silencieusement avant l'appel (à confirmer dans `signup.tsx`). Dans tous les cas, un utilisateur s'inscrivant juste avec un email se heurte soit à une erreur soit à un `username` choisi pour lui sans qu'il le sache.

5. Couverture API Moodle 4.4.3 - gaps fonctionnels

5.1 WS Moodle réellement utilisés

L'application invoque 32 web services Moodle distincts :

- `core_*`: `webservice_get_site_info`, `user_get_users_by_field`, `user_get_users`, `user_update_users`, `user_agree_site_policy`, `course_get_categories`, `course_get_courses`, `course_get_courses_by_field`, `course_get_contents`, `course_get_enrolled_courses_by_timeline_classification`, `enrol_get_users_courses`, `enrol_get_enrolled_users`, `badges_get_user_badges`, `completion_get_activities_completion_status`, `completion_get_course_completion_status`, `completion_update_activity_completion_status_manually`, `grades_update_grades`, `question_get_question_bank_entries`.

- `mod_*`: `quiz_get_user_attempts`, `quiz_start_attempt`, `quiz_get_attempt_data`, `quiz_get_attempt_access`, `quiz_get_quizzes_by_courses`, `quiz_save_attempt`, `quiz_process_attempt`, `assign_get_assignments`, `assign_save_submission`, `assign_submit_for_grading`, `assign_get_submissions`, `choice_get_choice_options`, `choice_submit_choice`, `choice_submit_choice_response`, `glossary_get_entries_by_letter`, `lesson_get_lesson`, `lesson_get_pages`, `lesson_get_lesson_attempts`, `lesson_process_page`, `lesson_finish_attempt`.
- `auth_email_signup_user`, `enrol_manual_enrol_users`.

Couverture quantitative : **32 fonctions sur ~600** disponibles dans Moodle 4.4.3. C'est cohérent avec une app verticalisée, mais plusieurs domaines critiques manquent.

5.2 Domaines Moodle 4.4.3 absents de l'intégration

Domaine	WS utiles non intégrés	Impact applicatif
Notes (gradebook)	<code>gradereport_user_get_grade_items</code> , <code>gradereport_user_get_grades</code> , <code>core_grades_get_grades</code> , <code>core_grades_get_grade_items</code>	l'élève ne peut pas consulter ses notes; l'enseignant n'a aucun retour.
Forum	<code>mod_forum_get_forums_by_course</code> , <code>mod_forum_get_forum_discussion_html</code> , <code>mod_forum_add_discussion</code> , <code>mod_forum_add_discussion_post</code>	Pas d'interaction sociale ni d'entraide pédagogique.
Messagerie	<code>core_message_send_messages_to_users</code> , <code>core_message_get_conversations</code> , <code>core_message_data_for_messagearea_*</code>	Pas de communication enseignant→élève.
Calendrier	<code>core_calendar_get_calendar_events</code> , <code>core_calendar_get_action_events</code>	Devoirs et échéances invisibles.
Recherche	<code>core_search_get_relevant_users</code> , <code>core_search_get_search_areas</code>	Pas de recherche transversale dans l'app.
Avatar utilisateur	<code>core_user_upload_user_picture</code> , <code>core_files_upload</code>	Le bouton "edit avatar" sur <code>edit-profile.tsx</code> n'a aucun backend.
Activités H5P	<code>mod_h5p_get_h5p_activities</code> , <code>mod_h5p_get_h5pactivity_access_urls</code>	H5P est très utilisé en pédagogie Moodle 4.4 - totalement ignoré.
Modules book/scorm/workshop	<code>mod_book_get_books_by_course</code> , <code>mod_scorm_get_scorms_by_course</code> , <code>mod_workshop_get_workshops_by_course</code>	Limite l'app à 5 types d'activités; toute autre est invisible.
Notifications mobiles	<code>message_airnotifier_*</code> , <code>tool_mobile_get_plugins_supported_plugins</code> , <code>core_user_get_private_files_info</code>	Aucune push; aucune notification mobile compatible mobile.
Restrictions / disponibilité	<code>core_course_get_contents</code>	Les cours conditionnels (« débloquer après finir le 1 ») ne sont pas appliqués.
Compétences / cadres	<code>core_competency_*</code>	Système Moodle 4.4 de compétences absent.

5.3 Custom fields `ipelan_*` non documentés

L'application pousse `ipelan_xp`, `ipelan_streak`, `ipelan_coins`, `ipelan_last_activity` via `core_user_update_users`. Aucune documentation ne confirme que ces *custom user info fields* existent côté Moodle. Si l'admin Moodle ne les a pas créés (Site administration > Users > User profile fields > Create a new profile field), tous les `update_users` échouent silencieusement et la gamification reste purement locale (SQLite). C'est cohérent avec le constat de `SYNTHESE_EXECUTIVE.md` (« XP/Streak/Profil jamais synchronisés »).

5.4 Service Moodle utilisé : confusion entre deux noms

- `services/api/moodleClient.ts:5:service: "IPELAN_FULL_SERVICE"` (jamais utilisé - variable morte).
- `services/api/moodleAuth.ts:43:service: "ipelan_full"` (réellement envoyé à `/login/token.php`).
- `.env: SERVICE_NAME=IPELAN_FULL_SERVICE` (jamais lu nulle part).

Trois noms de service différents cohabitent. À unifier : choisir un seul (`ipelan_full` semble être celui qui marche réellement) et le centraliser dans Config.

6. Fonctionnalités manquantes

6.1 Leaderboard - service implémenté, écran inexistant

Le service `services/api/leaderboardService.ts` est complet et lit bien `customfields.ipelan_xp / ipelan_streak` via `core_enrol_get_enrolled_users`. Mais aucun écran ne le consomme : il n'y a pas de fichier `app/(tabs)/(progress)/leaderboard.tsx` ni d'import de `getLeaderboard` dans l'arborescence `app/`. La fonctionnalité est citée dans le README et les onboardings mais elle est **invisible** dans l'app.

6.2 Téléchargement explicite pour offline

`services/sync/downloadService.ts` existe pour télécharger les EPUBs, mais aucun bouton « Télécharger pour hors ligne » n'apparaît dans `app/(tabs)/(cours)/index.tsx` ou `app/(stacks)/(cours)/[cours]`. Le téléchargement est implicite à l'ouverture de la leçon, ce qui rend impossible la préparation d'un usage hors-connexion (ex. en classe sans wifi).

6.3 Internationalisation (i18n)

Recherche `useTranslation|i18n|i18next` : **0 occurrence**. L'application enseigne le pulaar, le soninké et le wolof, et son interface est strictement en français. Pour des enfants du cycle fondamental dont le français peut être seconde langue, l'absence d'i18n est un défaut pédagogique majeur.

6.4 Accessibilité

Recherche `accessibilityLabel|accessibilityRole|accessibilityHint` : **0 occurrence** dans l'ensemble du dossier `app/` et `components/`. Aucun écran ne fournit de label aux lecteurs d'écran. L'app est donc non utilisable par un enfant malvoyant ou par un parent qui s'appuierait sur TalkBack/VoiceOver.

6.5 NetInfo et stratégie offline-first

Aucune dépendance `@react-native-community/netinfo` n'est installée (cf. `package.json`). La détection de connectivité utilise `fetch('https://www.google.com/favicon.ico', { method: 'HEAD', mode: 'no-cors' })` (`progressSync.ts:42`), ce qui est :

- **Lent** : timeout par défaut, plusieurs secondes en zone à connectivité dégradée.
- **Imprécis** : Google peut être joignable mais pas Moodle.
- **Inutilisable en réseaux corporate** où Google est filtré.

Une bibliothèque NetInfo officielle, doublée d'un *health check* sur `https://moodle.richatt.com/login/index.php` donnerait un résultat en ms et fiable.

6.6 Notifications push

Aucune dépendance expo-notifications ni expo-server-sdk. Aucun handler de message Moodle (message_airnotifier_*). L'app ne peut pas notifier les nouveaux devoirs, les badges remportés, les réponses du forum.

6.7 Renouvellement / révocation du token

Le code ne gère pas le 401/410 : si le token Moodle est révoqué côté admin, l'utilisateur reste « connecté » dans Redux et toutes les requêtes échouent en boucle sans provoquer de redirection vers /login. Ajouter un *interceptor* dans moodleFetch qui, sur errorcode === 'invalidtoken' ou accessexception, dispatch logout() et navigue.

6.8 Persistance Redux

Le store services/redux/store.ts n'utilise pas redux-persist. Le seul mécanisme de restauration est le hook custom useAuthRestore qui ne couvre que auth. Les autres slices (s'il y en avait pour XP, badges, courses) n'auraient aucune persistance.

6.9 Tests automatisés

Le dossier tests/ contient deux scripts ad-hoc (connexion.ts, lesson-test.ts) qui sont en réalité des outils de diagnostic à exécuter à la main avec un token admin. Aucun framework (Jest, Vitest, Detox) n'est configuré. Aucun test unitaire pour les *parsers* critiques (epubParserLite, quizService.parseQuestionHtml), aucun test e2e du flux login → cours → quiz → résultat.

6.10 CI/CD

Le dossier .github/ contient un répertoire java-upgrade mais aucun fichier .github/workflows/*.yaml. Aucune intégration continue : pas de tsc --noEmit automatique sur PR (ce qui aurait détecté §3.1), pas de lint, pas de build EAS. Un workflow basique épargnerait les régressions avant fusion.

6.11 Mode enseignant / parent

Le code suppose un rôle élève (roleid: 5 codé en dur dans enrolUserInCourse, services/api/moodleAuth.ts:12). Aucun écran de tableau de bord parent (suivi de progression de l'enfant) ni enseignant (assignation de devoirs, correction de dictées). Pour une vraie utilisation en classe, ces deux personas sont indispensables.

6.12 Gestion d'erreurs UX

Quand un fetch Moodle échoue, la majorité des hooks logue (console.warn) et retourne [] ou null. Le résultat à l'écran est un état vide ou un spinner permanent, sans message clair ni bouton de retry. À uniformiser via EmptyState (déjà disponible) et un toast d'erreur.

6.13 Validation côté client

Le formulaire de signup (app/(auth)/signup.tsx) valide le mot de passe (8 caractères, min/maj/chiffre) mais pas l'email (regex absente). La sanitisation input.replace(/[<>"'\\"/g, '') (ligne 42) supprime des caractères potentiellement légitimes (apostrophes dans les noms : O'Connor, etc.).

7. Architecture & dette technique

7.1 Triple couche d'accès Moodle redondante

Trois mécanismes coexistent pour appeler Moodle :

1. `services/api/moodleClient.ts` → `moodleFetch()` (le standard).
2. `services/contentLoader.ts` → `fetchWithAuth()` (avec gestion de token admin fallback).
3. `services/api/moodleActivities.ts` (autre couche encore avec ses propres helpers).

Les hooks et écrans piochent indifféremment dans les trois. La logique de fallback admin est répétée au moins six fois dans la codebase. À factoriser dans une seule classe `MoodleApi`.

7.2 Dépendance `react-native-sqlite-storage` doublée par `expo-sqlite`

`package.json` liste les deux : `expo-sqlite` (utilisée majoritairement) et `react-native-sqlite-storage` (avec ses types). Il n'y a qu'une seule base à gérer; supprimer l'une des deux dépendances.

7.3 NativeWind 2.0.11 sur RN 0.81 + React 19

`nativewind: 2.0.11` est antérieur au support officiel de React 19 / RN 0.81. Le passage à NativeWind v4 est recommandé pour ces versions, sous peine de bugs de rendu silencieux (classes ignorées en mode production).

7.4 Cache `moduleCache` jamais invalidé

`services/api/moduleResolver.ts:14` met en cache `getCourseModules()` au runtime. La fonction `clearModuleCache` existe mais n'est appelée nulle part. Si l'enseignant ajoute une activité, l'élève qui a la session ouverte ne la verra jamais. À invalider sur `pull-to-refresh`.

7.5 Tailles de fichiers - `unified-viewer.tsx`

`app/(stacks)/(cours)/content/unified-viewer.tsx` fait plus de 700 lignes et concentre PDF, EPUB, audio, vidéo. À découper en sous-composants (`PdfPane`, `EpubPane`, `MediaPane`).

8. Sécurité

Risque	Localisation	Sévérité
Token admin exposé dans le bundle	<code>.env</code> + 13 fichiers	Critique
<code>wstoken: ''</code> masqué en succès	<code>userProgressService.ts:273</code>	Élevée
<code>originWhitelist={['*']}</code> sur <code>WebView</code> avec <code>javascriptEnabled</code>	<code>EPUBLessonViewer.tsx:122</code> , <code>unified-viewer.tsx:528</code> , 700	Moyenne (si EPUB non maîtrisé, XSS)
Sanitisation regex naïve	<code>signup.tsx:42</code>	Faible mais corruptrice
Pas de <code>rate-limit</code> côté client	global	Faible
Pas d' <code>expirationTime</code> sur <code>SecureStore</code>	<code>tokenStorage.ts</code>	Faible

Recommandations spécifiques `WebView` :

- Restreindre `originWhitelist` à `['file://*', 'https://moodle.richatt.com']`.
- Désactiver `javascriptEnabled` sur EPUB (le HTML EPUB ne nécessite pas JS).
- Activer `setSupportMultipleWindows={false}`, `mixedContentMode='never'`.

9. Roadmap priorisée

[P0] Sprint 0 - Bloquants (1 semaine)

1. Corriger `wordOrderService.ts` (suppression du bloc dupliqué). Vérifier que `tsc --noEmit` passe.
2. Sortir le token admin du bundle : créer un endpoint proxy côté Moodle (plugin local PHP `local_ipelan_proxy`) qui accepte un token utilisateur et relaie vers les WS protégés.
3. Corriger la signature de `submitGradeToMoodle` (passer token en param) et propager dans `result.tsx`.
4. Unifier la clé `SecureStore` (`moodle_token` partout). Supprimer la chaîne magique `'auth_token'`.
5. Activer un workflow GitHub Actions minimal : `tsc --noEmit + expo lint` sur chaque PR.

[P1] Sprint 1 - Pédagogie réelle (2 semaines)

6. Migrer **listening** et **dictation** vers `mod_quiz` (questions audio + reconnaissance) plutôt que `mod_choice` / `mod_assign`. Réintégrer un `correctIndex` provenant réellement de Moodle.
7. Implémenter **reset password** via `core_auth_request_password_reset` (disponible Moodle 4.4) ou un lien externe `/login/forgot_password.php`.
8. Documenter et créer côté Moodle les **custom user fields** `ipelan_xp`, `ipelan_streak`, `ipelan_coins`, `ipelan_last_activity`; vérifier que `core_user_update_users` les accepte (ils doivent être en *visible to user*).
9. Brancher l'écran **Leaderboard** (création de `app/(tabs)/(progress)/leaderboard.tsx` et bouton vers cet écran depuis `(progress)/index.tsx`).
10. Intégrer **gradereport_user_get_grades_table** pour afficher les notes Moodle dans le profil.

[P2] Sprint 2 - Robustesse & UX (2 semaines)

11. Remplacer la détection réseau par `@react-native-community/netinfo`.
12. Ajouter `redux-persist` (slice auth + gamification + cours).
13. Ajouter intercepteur 401/invalidtoken dans `moodleFetch` → `dispatch logout()`.
14. Réduire le splash de 5 s à un *gate* responsive.
15. Remplacer tous les `Math.random() - 0.5` par Fisher-Yates (helper unique dans `utils/shuffle.ts`).
16. Migrer les 5 fichiers `StyleSheet.create` vers `NativeWind`, ou autoriser `StyleSheet` dans `AGENTS.md` si on l'assume.
17. Appliquer `i18n` minimal : extraire toutes les chaînes de `app/` vers un dictionnaire `i18n/fr.json`, puis ajouter `ar.json` (arabe local), `pu1.json`, `son.json`, `wol.json` au moins pour les libellés clés.

[P3] Sprint 3 - Fonctionnalités absentes (3 semaines)

18. **Forum** (`mod_forum_*`) - un onglet de discussion par cours.
19. **Messagerie** (`core_message_*`) - DM enseignant↔élève.
20. **Calendrier** (`core_calendar_*`) - vue agenda des devoirs.
21. **Notifications push** via expo-notifications + `message_airnotifier_*`.
22. **Mode parent** - onglet additionnel parent-dashboard (suivi enfants, restriction temps d'écran).
23. **Mode enseignant** - assignation rapide (`mod_assign_save_grade`), correction des dictées audio.
24. **H5P** (`mod_h5p_*`) pour activités interactives Moodle modernes.
25. Téléchargement explicite des cours (bouton «Télécharger ce cours pour hors-ligne») avec gestion de quota (`MAX_STORAGE_MB` déjà défini dans `constants/env.ts:21`).

[Q] Sprint 4 - Qualité (2 semaines)

26. Mise en place Jest + 30 tests unitaires sur les *parsers* (`parseQuestionHtml`, `parseOPFLite`).
27. Detox pour le e2e flux login → cours → quiz → résultat.

28. Audit accessibilité avec react-native-accessibility-engine; ajouter accessibilityLabel partout.
29. Sentry / Crashlytics pour la remontée d'erreurs en production.
30. Migrer NativeWind v2 → v4.

10. Annexes

10.1 Liste exhaustive des bugs détectés

#	Sévérité	Fichier	Ligne	Description
1	P0	services/api/word64483Service.ts		Code dupliqué hors scope, 3 erreurs TS bloquantes
2	P0	.env + bundle	-	Token admin exposé
3	P0	services/api/user270ProgressService.ts		token: ''
4	P0	services/sync/pro37essSync.ts		Mauvaise clé SecureStore
5	P1	app/(auth)/resetp24sword.tsx		Reset password = stub
6	P1	services/api/list60ingService.ts		Détournement de mod_choice
7	P1	services/api/quiz241vice.ts		Format de data: incohérent
8	P1	app/index.tsx	67	Splash 5 s bloquant
9	P1	8 fichiers	-	Math.random() - 0.5 biaisé
10	P1	app/(stacks)/(cou67)08listening.tsx		Audio simulé sans son
11	P1	app/(auth)/onboard35ng.tsx		Faute desXP
12	P1	5 fichiers	-	StyleSheet.create violant AGENTS.md
13	P1	hooks/useSignup.t14		Username obligatoire mais pas dans formulaire
14	P2	services/api/mood5eClient.ts		Config.service mort
15	P2	services/api/modul4Resolver.ts		Cache jamais invalidé
16	P2	services/redux/store.ts		Pas de redux-persist
17	P2	app/(stacks)/(cou528)700ent/unifiedviewer.tsx		ediginWhitelist={['*']} + JS
18	P2	app/(auth)/signup42sx		Sanitisation regex naïve
19	P2	package.json	-	Doublon expo-sqlite / react-native-sqlite-storage

#	Sévérité	Fichier	Ligne	Description
20	P2	package.json	-	NativeWind 2 sur RN 0.81/React 19

10.2 WS Moodle 4.4.3 prioritaires à intégrer

- gradereport_user_get_grades_table (notes)
- core_grades_get_grades (notes par activité)
- mod_forum_get_forums_by_courses, mod_forum_get_forum_discussions (forum)
- core_message_get_conversations, core_message_send_messages_to_conversation (messagerie)
- core_calendar_get_calendar_events, core_calendar_get_action_events_by_courses (agenda)
- core_user_upload_user_picture (avatar)
- core_auth_request_password_reset (reset password)
- mod_h5p_get_h5p_activities, mod_h5p_get_h5pactivity_access_information (H5P)
- mod_book_get_books_by_courses (book)
- core_search_get_relevant_users (recherche)
- tool_mobile_get_plugins_supporting_mobile (compatibilité plugins)
- message_airnotifier_* (push notifications)

10.3 Empreinte du projet

- Lignes de code : 14 192 (ts/tsx)
- Fichiers : 39 écrans + 5 composants + 30 services + 17 hooks + 7 types
- Dépendances de production : 36
- Dépendances de dev : 7
- Tests : 0 (deux scripts ad-hoc seulement)
- Couverture i18n : 0 %
- Couverture accessibilité : 0 %
- WS Moodle utilisés : 32
- Erreurs TypeScript actuelles : 3 (bloquantes)

10.4 Estimation d'effort par sprint

Sprint	Durée	Personne-jours	ROI
0 - Bloquants	1 semaine	5 j	App compile + sécurisée
1 - Pédagogie réelle	2 semaines	10 j	Activités vraiment connectées
2 - Robustesse & UX	2 semaines	10 j	Production-ready
3 - Fonctionnalités absentes	3 semaines	15 j	Couverture Moodle complète
4 - Qualité	2 semaines	10 j	Maintenable long terme
Total	10 semaines	50 j	App finie

11. Conformité au cahier des charges

Le cahier des charges officiel du stage IPELAN (sujet « Conception et développement d'une application mobile éducative interactive pour l'apprentissage des langues nationales en Mauritanie », durée 3-4 mois) impose un périmètre de **prototype**, pas de produit fini. Cette grille de lecture est essentielle pour pondérer les constats de l'audit : un défaut absent du cahier des charges (ex. i18n, mode parent, push) ne peut pas être qualifié de manquement contractuel — il devient une recommandation pour une éventuelle phase 2.

11.1 Exigences du cahier des charges - état réel

Exigence	État audité	Verdict cahier
Étudier contenus EPUB + audio existants	Pipeline EPUB présent (parserLite, unzipService, pathResolver); chapters navigation OK	Conforme
Architecture applicative multiplateforme	Expo SDK 54 + React Native 0.81, support Android/iOS/Web	Conforme
Prototype fonctionnel	Application démarre, login, listing cours, lecture EPUB OK	Conforme mais : ne compile pas en tsc --noEmit (§3.1)
Transformer contenus linéaires en parcours interactifs	learning-path.tsx + 5 activités	Partiellement conforme
Quiz interactifs	app/quiz/index.tsx + quizService	Conforme
Exercices d'association	app/(stacks)/(cours)/association.tsx + 'assoc	